Personalized Networking Assistant : AI -Powered Professional Context & Conversation Generator

Project Description:

The Personalized Networking Assistant harnesses Generative AI and Natural Language Processing to provide intelligent, context-aware networking preparation, offering users a fast and personalized icebreaker-creation experience. The platform includes an Event Analyzer module utilizing DistilBERT for zero-shot classification to extract core themes from event descriptions, a Topic Generator utilizing GPT-2 to produce tailored conversation starters, and a Wikipedia-powered Quick Verification Hub for rapid fact-checking.

Built with a decoupled architecture, the platform features a robust FastAPI backend for orchestrating AI models and data validation (via Pydantic), and a highly interactive Streamlit frontend for a seamless user experience. With secure local telemetry logging for session history and user feedback, the assistant empowers professionals and students to craft compelling networking strategies with confidence..

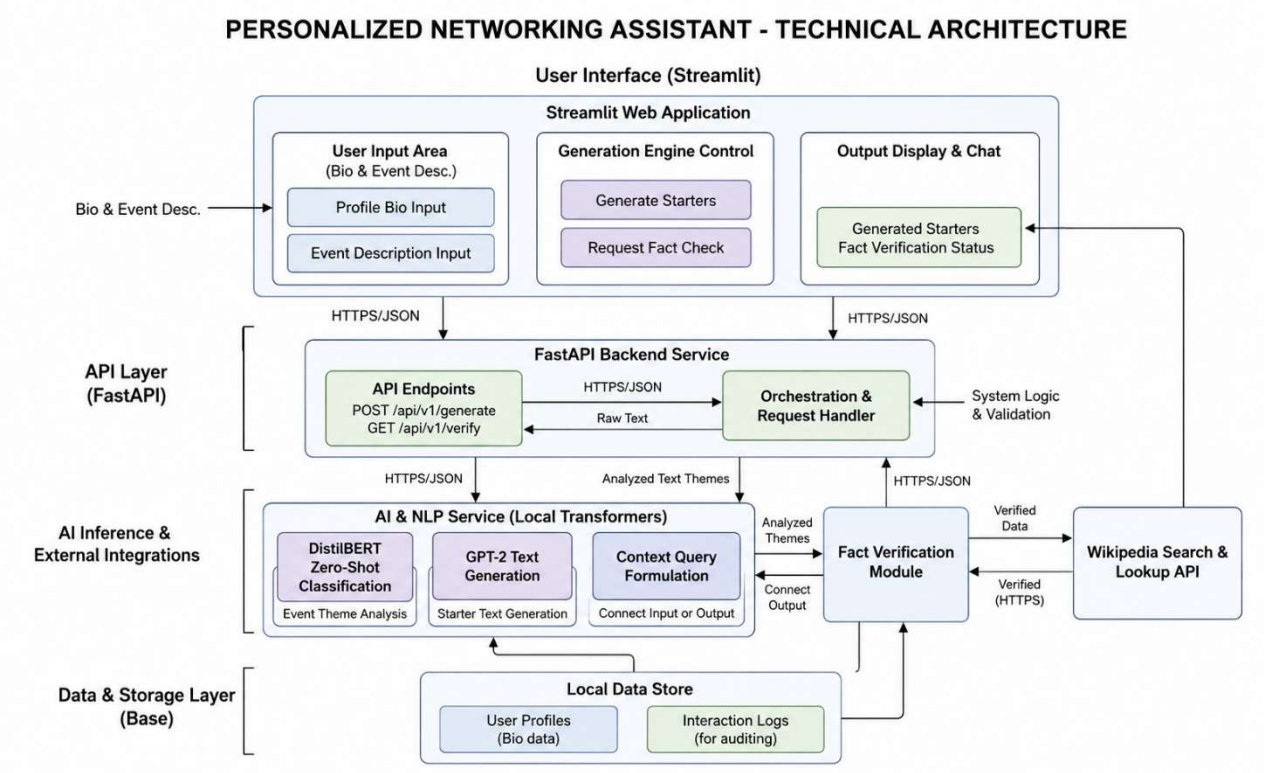
Scenarios

Scenario 1: A user is attending a global tech summit focusing on artificial intelligence and cybersecurity. They input the event description and list their personal interests as "AI, Startups, Networking." The system's DistilBERT model extracts the core themes, and the GPT-2 model generates three highly contextual, punchy conversation starters bridging the event's focus with the user's specific startup interests.

Scenario 2: While reviewing their generated icebreakers for a healthcare networking event, a user encounters an ambiguous medical-tech term they want to understand better before the event. Instead of leaving the app, they use the Quick Verification Hub, which instantly pings the Wikipedia REST API and returns a concise, structured summary of the concept.

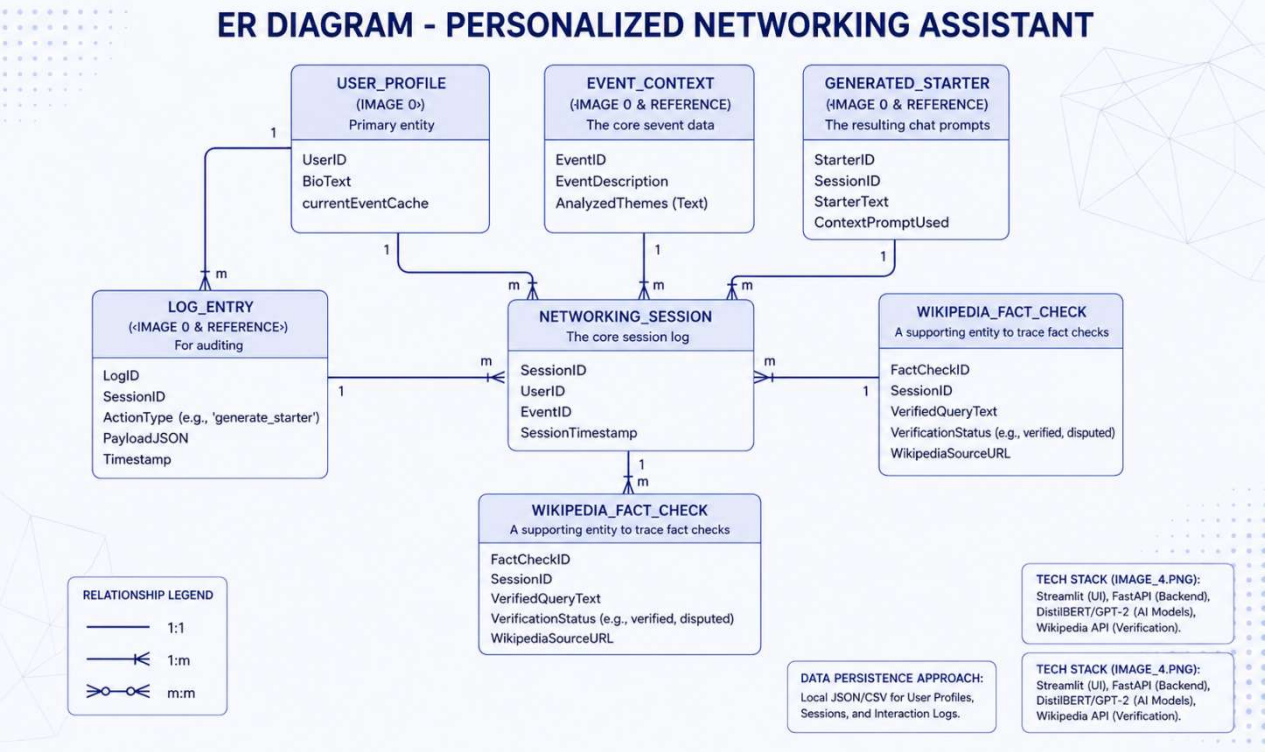
Scenario 3: A user wants to review their previous networking preparation. They access the local history and telemetry view within the application, allowing them to see their past event prompts, the AI-generated responses, and whether they had explicitly "Liked" or "Disliked" specific suggestions, creating a feedback loop for personal reflection.

Technical Architecture:



Description: The Personalized Networking Assistant utilizes a modular three-tier architecture to deliver rapid, AI-driven context generation. The frontend provides a responsive, state-preserved user interface built entirely in Python using Streamlit. The FastAPI-based backend orchestrates strict type-checking data validation (Pydantic) and prompt construction. It interfaces natively with Hugging Face transformers (DistilBERT and GPT-2 loaded at startup for efficiency) and external web services (Wikipedia API), with persistent local storage handled via JSON file operations.

ER Diagram:



Description:

ER Diagram (ERD) is a visual representation of a database's logical structure, defining entities, attributes, and relationships. It optimizes data organization, supports normalization and integrity, and simplifies query design.

Pre-requisites:

- **Python Programming Proficiency**: Python 3.10+ documentation and virtual environment management.
- **FastAPI Framework Knowledge:** FastAPI Documentation for asynchronous routing and Pydantic validation.
- **Streamlit UI Familiarity:** Streamlit documentation for reactive state management (st.session_state).
- **Hugging Face Transformers:** Understanding of NLP pipelines, zero-shot classification, and text- generation.
- **Automated Testing:** Pytest framework and httpx TestClient for route validation.

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Set up the development environment, creating a virtual environment and installing dependencies from requirements.txt (FastAPI, Streamlit, transformers, torch).
- **Activity 1.2:** Research and select DistilBERT for zero-shot theme classification due to its balance of inference speed and accuracy.
- **Activity 1.3:** Select GPT-2 Small for natural text generation, ensuring it runs efficiently on standard hardware without GPU acceleration.
- **Activity 1.4:** Define the three-tier application architecture separating the Streamlit frontend, FastAPI routing layer, and AI service modules.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop Pydantic data contract validation models (e.g., ConversationRequest, FactCheckResponse) to enforce strict schema boundaries.
- **Activity 2.2:** Implement the AI service modules: event_analyzer.py (DistilBERT) and topic_generator.py (GPT-2 prompt engineering).
- **Activity 2.3:** Implement external and local services: fact_checker.py for Wikipedia API requests, and local JSON loggers for history and feedback.

Milestone 3: FastAPI Routing Orchestration

- **Activity 3.1:** Write the main routing logic in routers/conversation.py, establishing POST endpoints for /analyze-event, /fact-check, and /generate-conversation.
- **Activity 3.2:** Configure main.py as the application entry point, attaching routers and generating the interactive Swagger UI documentation.

Milestone 4: Frontend Development using Streamlit

- **Activity 4.1:** Design the user interface in frontend/app.py, creating input fields for event descriptions and user interests.
- **Activity 4.2:** Implement st.session_state to handle dynamic content rendering for API responses, displaying AI-generated captions and handling interactive "Like/Dislike" feedback buttons.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Write comprehensive automated tests using Pytest and httpx TestClient, mocking external network calls to achieve >80% coverage.
- **Activity 5.2:** Conduct performance and load testing using Locust to simulate concurrent virtual users against the backend endpoints.
- **Activity 5.3:** Deploy the application locally, launching the FastAPI server on port 8000 and the Streamlit frontend on port 8501.

Milestone 1: Model Selection and Architecture

In this milestone, we focus on establishing the environment and selecting the most efficient NLP models that can be run natively without requiring external API credits.

Activity 1.1: Set Up the Development Environment

- **Create and activate a virtual environment:** Set up a virtual environment using venv to isolate project dependencies:

```
python -m venv venv
venv\Scripts\activate
```

- **Install Dependencies:** Install all core dependencies including torch, transformers, fastapi, and streamlit:

```
pip install -r requirements.txt
```

- **Start Backend:** Run the backend server using uvicorn. This starts the API on port 8000 by default.

```
uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

- **Start Frontend:** Open a new terminal window or tab, activate the virtual environment, and execute:

```
streamlit run frontend/app.py
```

Activity 1.2 & 1.3: Model Selection & Instantiation

DistilBERT was selected for its balance of speed and zero-shot classification capabilities, while GPT-2 Small provides optimal offline generation speed. Crucially, models are instantiated at the module level in config.py so they are loaded into memory exactly once at application startup, significantly reducing latency on subsequent user requests.

```
from transformers import pipeline

# Global pipeline initialization
classifier = pipeline("zero-shot-classification", model="distilbert-base-uncased-mnli")
generator = pipeline("text-generation", model="gpt2")
```

Activity 1.4: Define Application Architecture:

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components like the frontend (Streamlit), backend (FastAPI), and local AI/API integrations.
- **Detail Frontend Functionality:** Outline how users interact with the application by entering event descriptions and personal interests, and providing explicit feedback (Like/Dislike).
- **Outline Backend Responsibilities:** Specify how the FastAPI backend handles incoming POST requests to /generate-conversation, validates input via Pydantic schemas, and orchestrates the AI modules.
- **Describe AI Integration Points:** Define how the backend uses DistilBERT for zero-shot theme extraction, constructs prompts for GPT-2 to generate tailored icebreakers, and returns structured JSON results to the frontend.

Milestone 2: Core Functionalities Development

Activity 2.1: Data Contract Validation

FastAPI uses Pydantic to ensure all incoming data conforms to strict schemas. This replaces hardcoded values with safely parsed, user-defined inputs from the frontend interface.

```
from pydantic import BaseModel
from typing import List

class ConversationRequest(BaseModel):
    description: str
    interests: List[str]
```

Activity 2.2: Topic Generator Integration

The generate_topics function accepts dynamic, user-defined inputs and weaves them into a structured prompt for GPT-2, followed by post-processing logic to enforce concise outputs.

```
def generate_topics(event_themes, user_interests):
    prompt = (
        f"I'm attending a networking event focused on {', '.join(event_themes)}. "
        f"I'm personally interested in {', '.join(user_interests)}. "
        f"What are three creative conversation starters I could use?"
    )
    outputs = generator(prompt, max_length=80, num_return_sequences=1)
    raw_text = outputs[0]["generated_text"]
    return [s.strip("-").strip() for s in raw_text.split("\n")[:3] if s.strip()]
```

Milestone 3: FastAPI Routing Orchestration

Milestone 3 focuses on creating and refining the core logic of the API router and entry point, which serves as the backbone of the application. Here we set up the endpoints, establish reliable model interaction, and automatically log the history.

Activity 3.1: Writing the Main Routing Logic

- **Define API Endpoints:** Established three independent POST routes using APIRouter(): /analyze- event (theme extraction), /fact-check (Wikipedia queries), and /generate-conversation (core orchestration).
- **Service Orchestration:** In /generate-conversation, the route acts as a controller: it first passes data to the event_analyzer to fetch themes, feeds those themes to the topic_generator, and captures the generated output.
- **Side-Effect Logging:** Implemented automatic telemetry logging. Every successful conversation generation silently triggers history_logger.log_conversation() to persist the session data locally.
- **Response Model Enforcement:** Bound response_model=ConversationResponse to the route to ensure outgoing serialized JSON strictly adheres to the frontend contract.

```
from fastapi import APIRouter
from app.models.schemas import ConversationRequest, ConversationResponse
from app.services import event_analyzer, topic_generator, history_logger

router = APIRouter()

@router.post("/generate-conversation", response_model=ConversationResponse)
def generate_conversation(data: ConversationRequest):
    # 1. Extract themes using DistilBERT
    themes = event_analyzer.extract_event_themes(data.description)

    # 2. Generate icebreakers using GPT-2
    suggestions = topic_generator.generate_topics(themes, data.interests)

    # 3. Background Telemetry: Save the session context automatically
    history_logger.log_conversation({
        "description": data.description,
        "interests": data.interests,
        "topics": themes,
        "suggestions": suggestions
    })

    return ConversationResponse(topics=themes, suggestions=suggestions)
```

Activity 3.2: Application Entry Point & Infrastructure Configuration

- **Configure main.py:** Instantiated the core FastAPI application object with a descriptive title for auto- generated documentation.
- **Register Routers:** Linked the isolated conversation router into the main app via app.include_router(), supporting a hub-and-spoke module architecture for future scale.
- **Health-Check Endpoint:** Added a root GET / endpoint to allow rapid verification of server uptime prior to running intensive NLP tasks.
- **Swagger UI Generation:** Enabled automatic OpenAPI 3.0 documentation served at /docs for interactive, visual testing of the Pydantic schemas.

```
from fastapi import FastAPI
from app.routers import conversation

app = FastAPI(title="Personalized Networking Assistant")

# Register modular routes
app.include_router(conversation.router)

# Health-check endpoint
@app.get("/")
def root():
    return {"message": "Welcome to the Networking Assistant API!"}
```

Milestone 4: Frontend Development using Streamlit

Milestone 4 focuses on developing a user-friendly interface. Using Streamlit, we design a seamless, single- page application that handles user inputs and renders AI-generated content dynamically without requiring page reloads.

Activity 4.1: Designing the User Interface

We set up the base structure in frontend/app.py, adding headers and input fields to capture the event description and user interests.

```
import streamlit as st
import requests

st.set_page_config(page_title="Networking Assistant", layout="centered")
st.title("✨ Personalized Networking Assistant")

# Input fields for dynamic user context
event_desc = st.text_area("Event Description", placeholder="Type the event details here...")
user_interests = st.text_input("Your Interests", placeholder="AI, Blockchain, Cyber Security")
```

Activity 4.2: Dynamic Content Rendering and State Management

Because Streamlit re-runs the entire script on every interaction, we utilize st.session_state to preserve the API response data. We dynamically loop through the generated suggestions, creating interactive "Like" and "Dislike" buttons for each item that ping the telemetry logger.

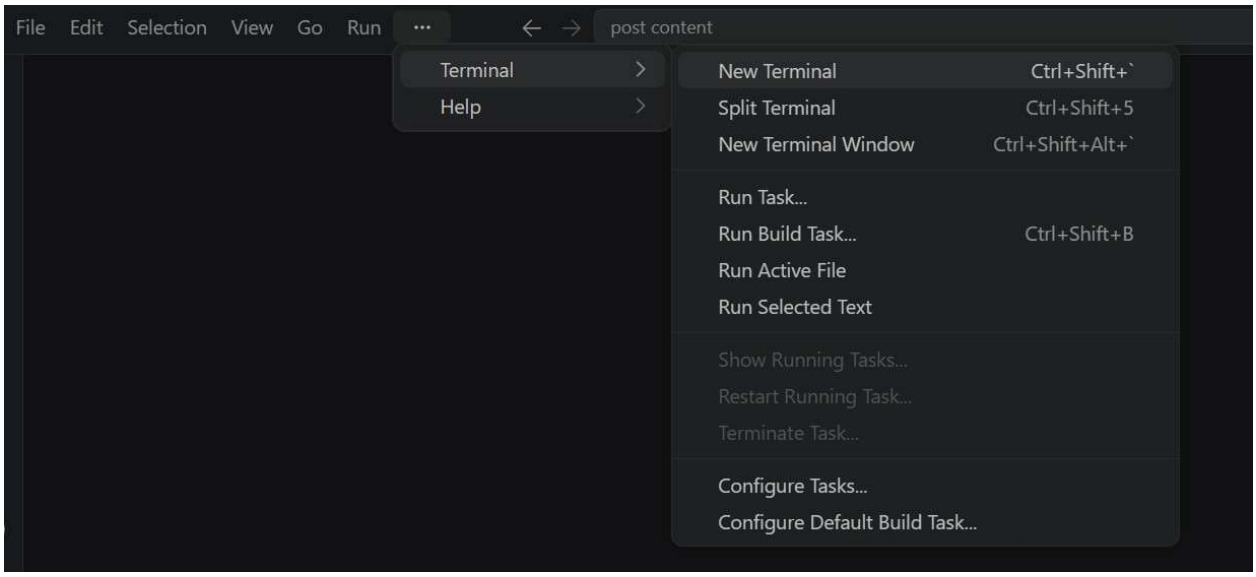
```
if st.button("Generate Conversation Starters"):
    if event_desc and user_interests:
        # Clean user inputs
        parsed_interests = [i.strip() for i in user_interests.split(",") if i.strip()]
        payload = {"description": event_desc, "interests": parsed_interests}

        # Call the FastAPI backend
        res = requests.post("http://127.0.0.1:8000/generate-conversation", json=payload)
        if res.status_code == 200:
            data = res.json()
            # Store results in session state to prevent disappearance on re-renders
            st.session_state["topics"] = data.get("topics", [])
            st.session_state["suggestions"] = data.get("suggestions", [])

# Render results dynamically if they exist in the session state
if "suggestions" in st.session_state:
    st.subheader("💡 Tailored Icebreakers")
    for i, item in enumerate(st.session_state["suggestions"]):
        st.info(item)

        # Interactive Layout for explicit user feedback
        col1, col2 = st.columns([1, 1])
        if col1.button(f"👍 Like", key=f"like_{i}"):
            feedback_logger.log_feedback(item, "like")
            st.toast("Feedback registered!")
        if col2.button(f"👎 Dislike", key=f"dislike_{i}"):
            feedback_logger.log_feedback(item, "dislike")
            st.toast("Feedback registered!")
```

Milestone 5: Testing and Deployment

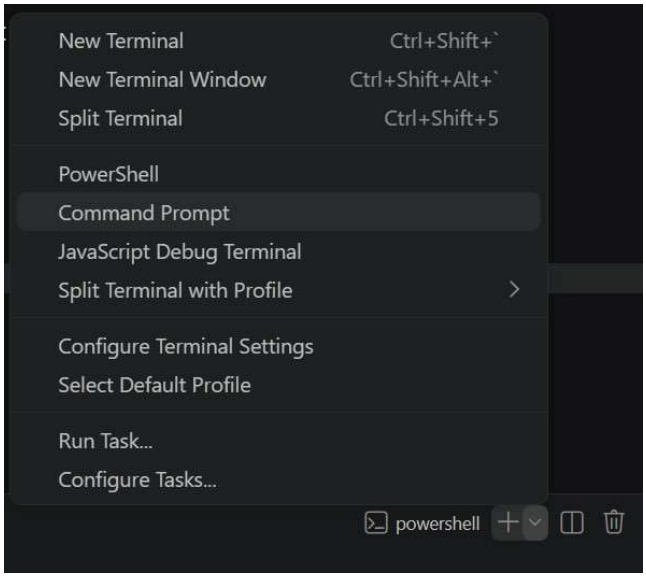
Activity 5.1: Functional and Unit Testing:

Step 1: Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
pip install -r requirements.txt
```

```
python -m venv venv
venv\Scripts\activate
```

Step 2:

The project uses pytest to run its test suites, validating routing, generation, fact-checking, and file logging logic.

```
# Run the entire test suite with coverage computation:
pytest --cov=app tests/

# Note: On Windows, if execution policies block running the pytest executable directly,
# run it via the Python module invocation instead:
python -m pytest --cov=app tests/

# To run a specific test file (e.g., event analyzer tests):
python -m pytest tests/test_event_analyzer.py
```

Activity 5.2: Performance Load Testing

This project contains a locustfile.py to benchmark performance under high concurrency.

```
# 1. Install locust:
pip install locust

# 2. Start the FastAPI backend server first.
# 3. Start the Locust runner:
locust -f locustfile.py

# 4. Open the Locust web interface at http://localhost:8089 and configure:
#    - Number of users: e.g., 10
#    - Spawn rate: e.g., 1
#    - Host: http://127.0.0.1:8000
```

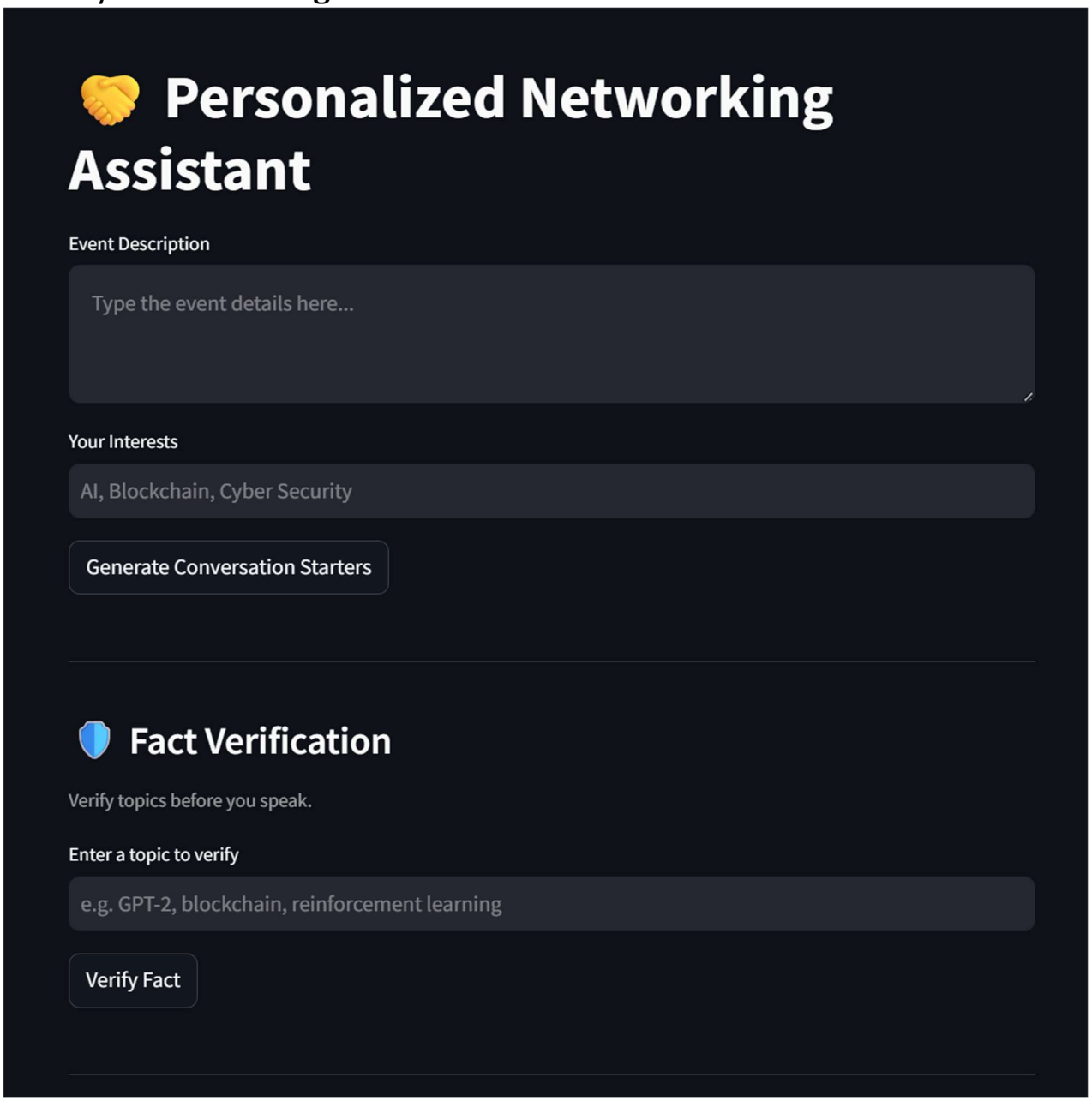
Activity 5.3: Application Boot Sequence

The decoupled architecture requires the server and client to run simultaneously across two ports.

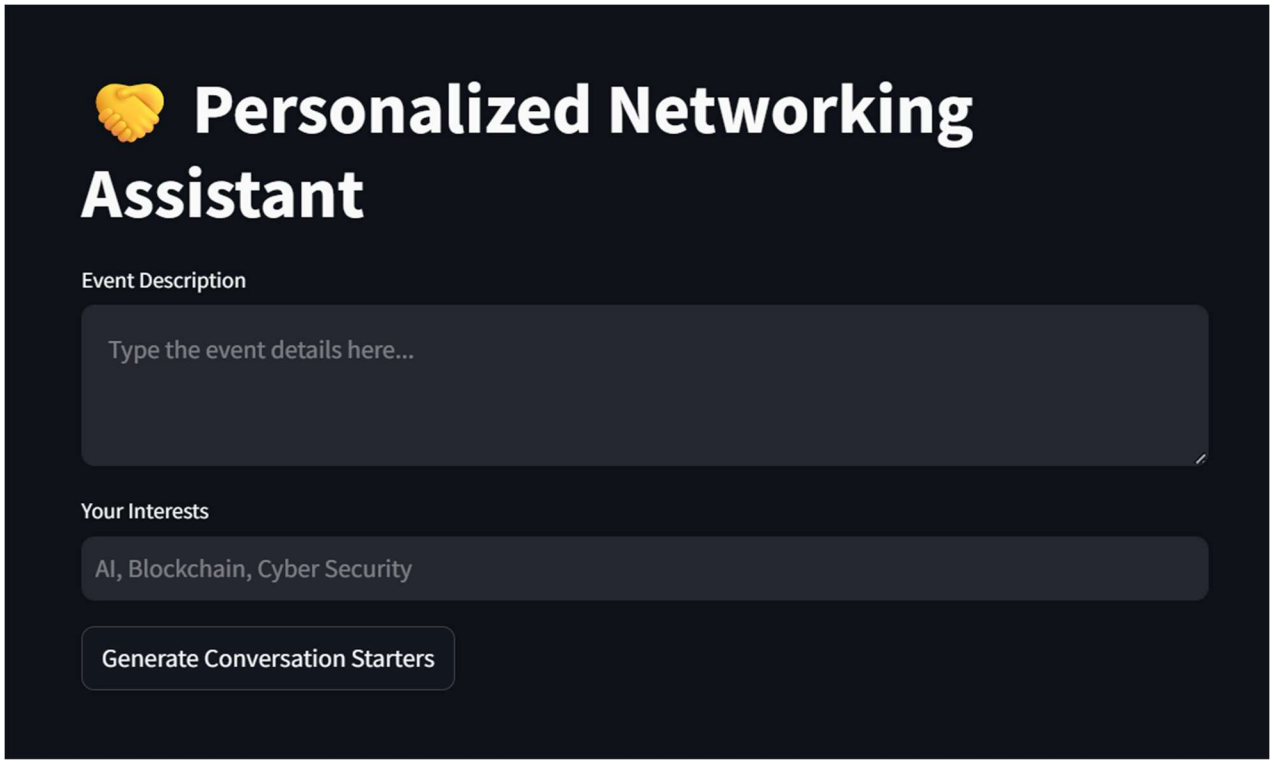
```
# Terminal 1 (Backend API)
uvicorn app.main:app --reload --port 8000

# Terminal 2 (Frontend Dashboard)
streamlit run frontend/app.py --server.port 8501
```

Exploring the Website's Web Pages:

Home / Generator Page:

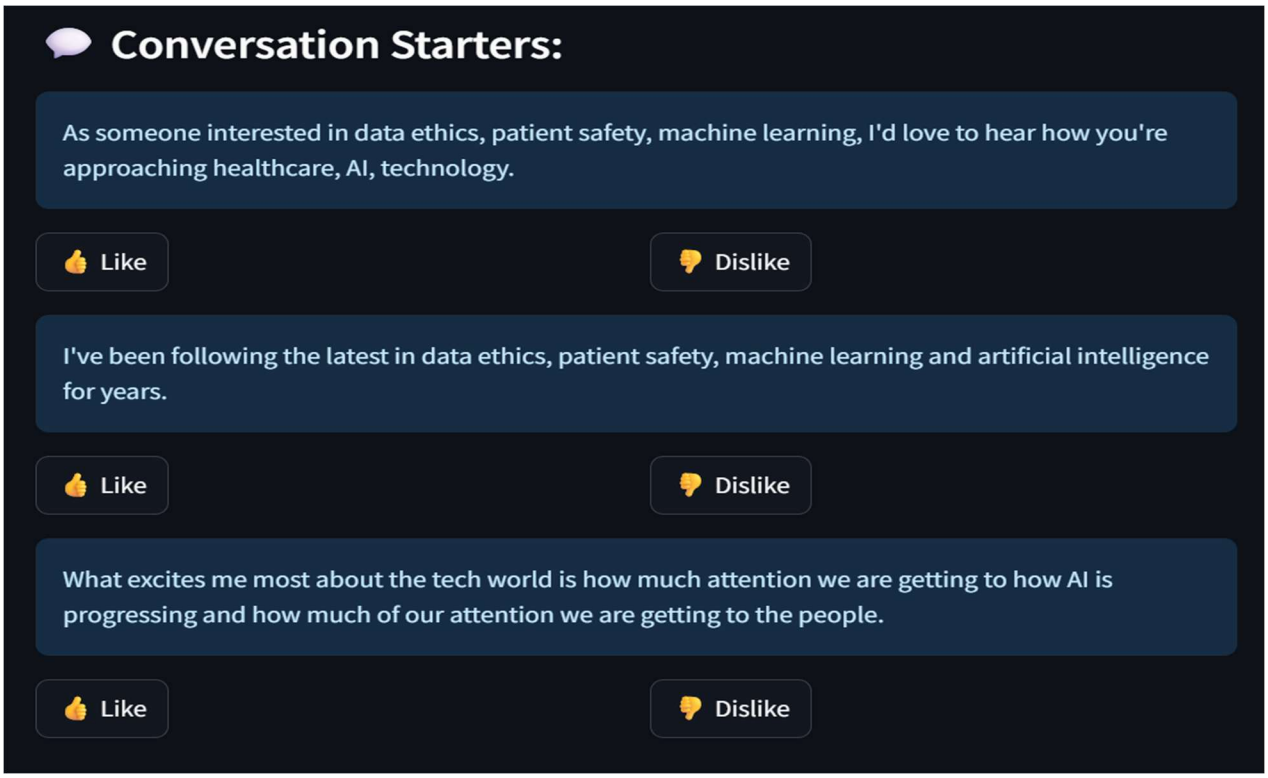


Input Section:

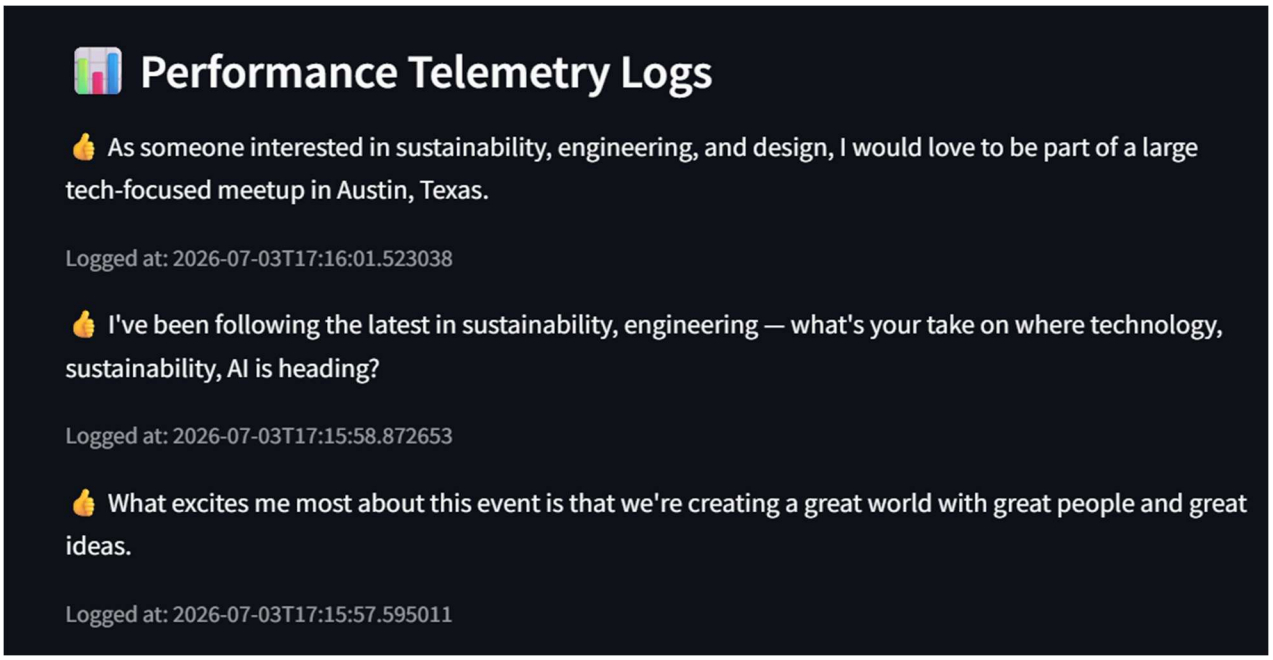


Input Section: The streamlined single-page interface serves as the primary generation tool. It features direct text areas for users to paste their target "Event Description" and a comma-separated list of "Your Interests." The interface is highly responsive; when the "Generate Conversation Starters" button is clicked, a JSON payload is securely routed to the backend API.

Dynamic Results :

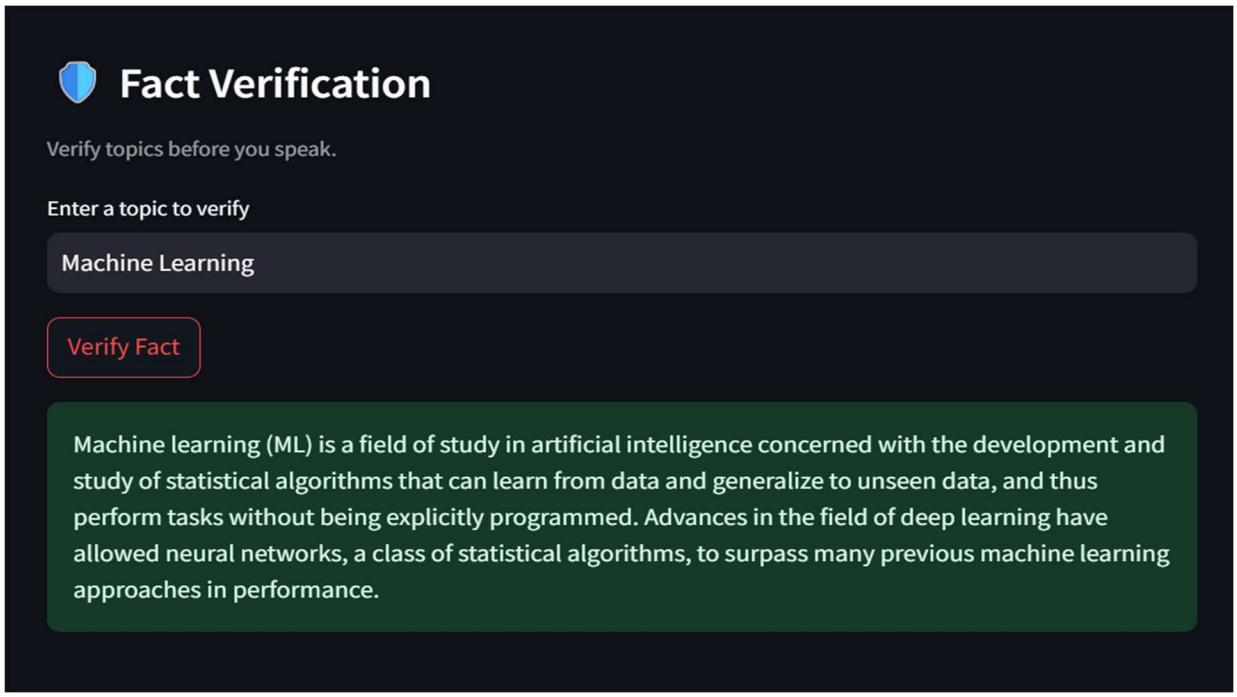


Telemetry Logs:

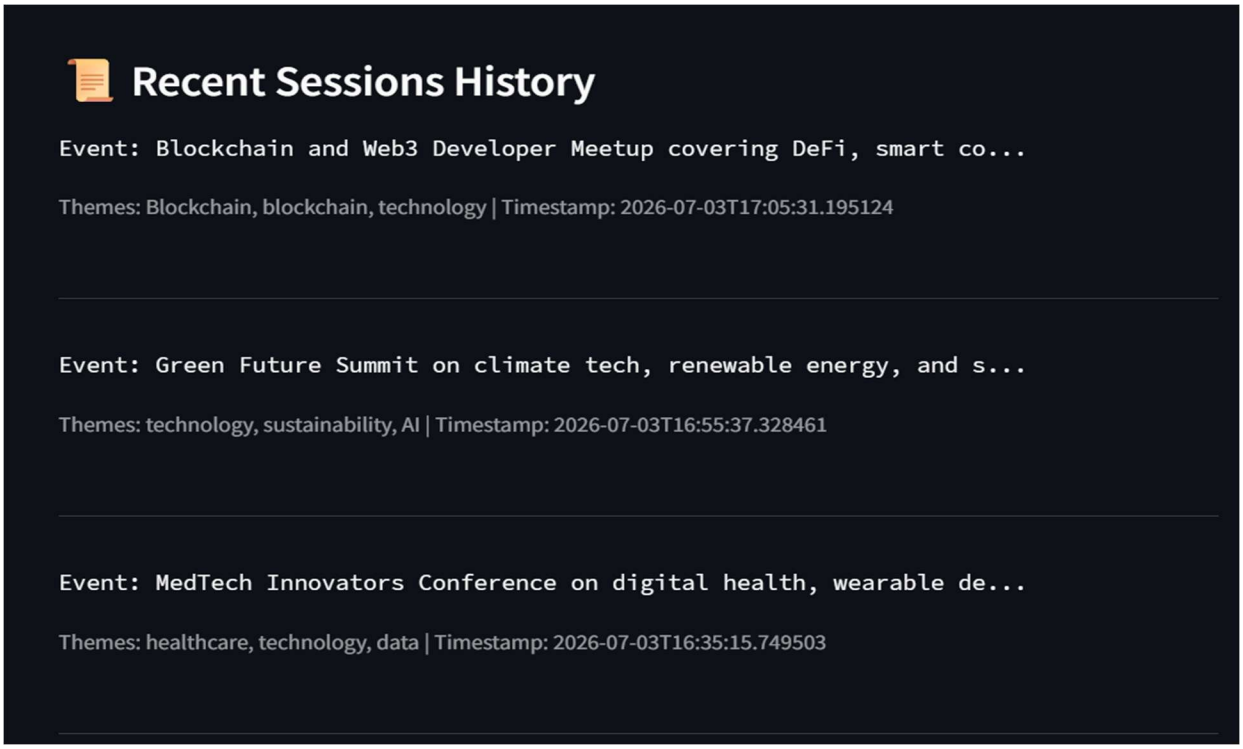


Results Section: Rendered dynamically using Streamlit's session state, this section displays the extracted "Event Focus Themes" and three "Tailored Icebreakers." Each icebreaker is accompanied by explicit "Like" and "Dislike" buttons, which fire background telemetry events to the local storage logger for future prompt refinement.

Verification:



Sessions History:



Utility Section: The bottom half of the workspace features the "Wikipedia Quick Verification Hub," allowing users to instantly clarify terminology without losing their place. Below this, the "Recent Sessions History" and "Performance Telemetry Logs" render visual historical context parsed directly from the backend's local JSON stores

Conclusion:

The Personalized Networking Assistant is a highly modular platform built with FastAPI and Streamlit that dramatically streamlines professional networking preparation. By leveraging Hugging Face's local transformer models (DistilBERT and GPT-2), it instantly generates tailored conversational contexts and themes without relying on paid, cloud-based LLM APIs. The project highlights how strict Single Responsibility Principles, robust automated testing (Pytest), and structured data contracts (Pydantic) can create highly resilient AI applications.

Looking ahead, the Networking Assistant offers significant opportunities for enterprise expansion, such as migrating the local JSON loggers to a robust Cloud SQL relational database, integrating full OAuth user authentication for personalized multi-device session tracking, and containerizing the architecture via Docker for scalable cloud deployment.